

# Projet base de données

## Projet 1 : LOGICIEL DE GESTION DE PROJETS

### PROJET DE BASE DE DONNEES

ENSAE 2026 : ISE2

Sous la supervision de : Baye Demba DIACK

Réalisé par : Khadidiatou COULIBALY, Dior MBENGUE, Kerencia Dyvana SEUNKAM PAHANE



## Analyse fonctionnelle du système

### Contexte

Plusieurs institutions publiques et organisations non gouvernementales souhaitent centraliser la gestion de leurs projets, actuellement dispersée entre fichiers Excel, échanges informels et outils hétérogènes. Cette fragmentation entraîne des retards, des doublons d'information et une visibilité limitée sur l'avancement réel des actions menées.

Face à ces limites, la mise en place d'une plateforme nationale unique apparaît comme une nécessité stratégique. Celle-ci doit offrir un espace collaboratif, sécurisé et structuré permettant de suivre en temps réel les projets, les tâches, les équipes, les documents et les interactions associées.

L'objectif global de cette plateforme est d'harmoniser les pratiques de gestion, de faciliter la coordination entre les différentes organisations et de garantir un pilotage fiable des projets de développement. Elle a vocation à devenir un référentiel commun soutenant la prise de décision et améliorant l'efficacité collective.

--

### Acteurs

## Administrateur

L'administrateur supervise l'ensemble de la plateforme et veille à son bon fonctionnement. Il gère les comptes utilisateurs, leurs rôles ainsi que les équipes. Il dispose d'une visibilité globale sur tous les projets et peut intervenir en cas de problème d'organisation ou de droits d'accès.

## Chef de projet

Le chef de projet est responsable de la planification, de la coordination et du suivi des projets relevant de son équipe. Il crée les projets, organise les tâches et répartit le travail entre les membres. Il suit l'avancement des activités, ajuste les priorités et anime la collaboration au sein de l'équipe.

## Membre d'équipe

Le membre d'équipe participe directement à la réalisation des tâches qui lui sont attribuées. Il met à jour l'état d'avancement de ses tâches, échange avec les autres utilisateurs via les commentaires et consulte les informations relatives aux projets. Son rôle est d'exécuter les activités opérationnelles et de contribuer au bon déroulement des projets.

--

# 3. Fonctionnalités

## Gestion des utilisateurs et des équipes

La plateforme permet de créer des utilisateurs, de définir leurs rôles et de les organiser au sein d'équipes. Chaque utilisateur appartient à au moins une équipe, ce qui structure la collaboration et la répartition du travail. L'administrateur contrôle les accès et veille à ce que chaque équipe dispose des membres nécessaires.

## Gestion des projets

La gestion des projets permet de créer, modifier et suivre les projets d'une équipe. Chaque projet est associé à une seule équipe et contient des informations essentielles telles que le budget, les dates et le statut. Le chef de projet peut y associer plusieurs utilisateurs afin d'assurer le pilotage et la réalisation des activités.

## Gestion des tâches

Le système permet de créer des tâches, de définir leurs priorités et de les assigner à un ou plusieurs membres. Chaque tâche est obligatoirement rattachée à un projet et possède un statut ainsi qu'un pourcentage d'avancement. Une tâche peut également rester non assignée lorsqu'elle est placée en backlog.

## Gestion des activités (logs)

Chaque action réalisée sur une tâche génère automatiquement une activité, telle qu'un commentaire, un changement de statut ou une réassignation. Ces activités permettent de conserver un historique complet et horodaté des actions effectuées. Elles assurent la traçabilité et facilitent le suivi du travail.

## Gestion des documents

La plateforme permet d'ajouter des documents associés aux projets ou aux tâches. Chaque document est caractérisé par un auteur, une date d'upload et un type, et doit être rattaché soit à un projet, soit à une tâche. Cette règle garantit une organisation claire des fichiers et évite les documents non référencés.

## Notifications (optionnel)

Le système peut générer des notifications lorsqu'un événement important survient, tel que la création d'une nouvelle tâche ou l'ajout d'un commentaire. Chaque notification est destinée à un utilisateur spécifique et peut être marquée comme lue ou non lue. Ce mécanisme permet aux membres de rester informés en temps réel.

--

## Règles de gestion

Les règles de gestion suivantes définissent les principes de fonctionnement de la plateforme et encadrent l'utilisation des différents objets du système.

### Règles liées aux utilisateurs

- **RG1** : Chaque utilisateur est identifié de manière unique dans le système.
- **RG2** : Un utilisateur possède un nom, un prénom, un email et un rôle.
- **RG3** : Un utilisateur peut avoir un rôle parmi : Administrateur, Chef de projet ou Membre.
- **RG4** : La date d'inscription d'un utilisateur est enregistrée.

### Règles liées aux équipes

- **RG5** : Chaque équipe est identifiée de manière unique.
- **RG6** : Une équipe possède un nom, une description et une date de création.
- **RG7** : Une équipe regroupe un ou plusieurs utilisateurs.
- **RG8** : Un utilisateur peut appartenir à une ou plusieurs équipes.
- **RG9** : L'appartenance d'un utilisateur à une équipe est caractérisée par une date d'affectation.

### Règles liées aux projets

- **RG10** : Chaque projet est identifié de manière unique.
- **RG11** : Un projet possède un titre, une description, des dates, un budget, un statut et une priorité.
- **RG12** : Chaque projet est géré par une seule équipe.
- **RG13** : Une équipe peut gérer un ou plusieurs projets.

## Règles liées aux tâches

- **RG14** : Chaque tâche est identifiée de manière unique.
- **RG15** : Une tâche possède un titre, une description, des dates prévues, une priorité et un pourcentage d'avancement.
- **RG16** : Chaque tâche appartient obligatoirement à un seul projet.
- **RG17** : Un projet peut contenir zéro, une ou plusieurs tâches.

## Règles liées à l'assignation des tâches

- **RG18** : Une tâche peut être assignée à zéro, un ou plusieurs utilisateurs.
- **RG19** : Un utilisateur peut être assigné à zéro, une ou plusieurs tâches.
- **RG20** : L'assignation d'un utilisateur à une tâche est caractérisée par une date d'affectation et un rôle dans la tâche.

## Règles liées aux activités (logs)

- **RG21** : Chaque activité est identifiée de manière unique.
- **RG22** : Une activité possède un contenu, une date/heure et un type.
- **RG23** : Une activité est réalisée par un seul utilisateur.
- **RG24** : Une activité est associée à une seule tâche.
- **RG25** : Une tâche peut générer zéro, une ou plusieurs activités.

## Règles liées aux documents

- **RG26** : Chaque document est identifié de manière unique.
- **RG27** : Un document possède un nom, un type, une date d'upload et un chemin de fichier.
- **RG28** : Un document est ajouté par un utilisateur.
- **RG29** : Un document est rattaché soit à un projet, soit à une tâche, mais jamais aux deux simultanément.
- **RG30** : Un projet ou une tâche peut avoir zéro, un ou plusieurs documents.

## Règles liées aux notifications

- **RG31** : Chaque notification est identifiée de manière unique.
- **RG32** : Une notification possède un message, un type, un statut et une date/heure.
- **RG33** : Une notification est reçue par un seul utilisateur.
- **RG34** : Un utilisateur peut recevoir zéro, une ou plusieurs notifications.

# Exigences non fonctionnelles

## Sécurité

L'accès à la plateforme est contrôlé par des rôles et par l'appartenance aux équipes et aux projets. Les utilisateurs ne peuvent consulter ou modifier que les informations auxquelles ils sont autorisés.

## Traçabilité

Toutes les actions importantes réalisées sur les tâches sont enregistrées dans un historique. Chaque action est associée à un utilisateur et à une date afin d'assurer le suivi et la responsabilité des interventions.

## Collaboration

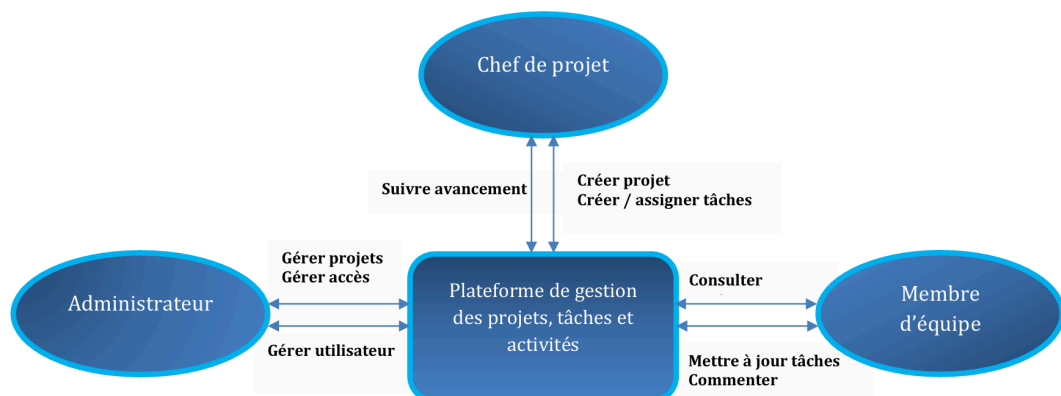
La plateforme doit permettre à plusieurs utilisateurs de travailler simultanément sur un même projet. Les membres peuvent échanger via des commentaires et suivre l'évolution des tâches en temps réel.

--

## Diagramme de contexte (description)

Le diagramme de contexte présente la plateforme de gestion de projets comme un système central en interaction avec trois acteurs principaux : l'administrateur, le chef de projet et le membre d'équipe.

Ces acteurs échangent avec le système pour créer, consulter et mettre à jour des informations relatives aux projets, aux tâches, aux documents et aux activités. La plateforme centralise l'ensemble de ces échanges et garantit un accès contrôlé selon les rôles et les droits des utilisateurs. Le schéma suivant



--

## Uses cases (Scénarios d'utilisation)

Cette partie présente les scénarios d'utilisation (use cases) de la plateforme de gestion des projets, tâches et activités. Les use cases décrivent, de manière concrète, les différentes interactions possibles entre les acteurs du système et la plateforme, en fonction de leurs rôles et responsabilités. Ils permettent de traduire les besoins fonctionnels en situations réelles d'utilisation, et constituent une base essentielle pour la compréhension du fonctionnement global du système ainsi que pour la modélisation de la base de données.

## **Acteur : Administrateur**

- UC1 : Créer un utilisateur

L'administrateur crée un nouvel utilisateur en renseignant ses informations personnelles (nom, prénom, email, téléphone) et en lui attribuant un rôle. L'utilisateur est ensuite enregistré dans le système et peut accéder à la plateforme selon ses droits.

- UC2 : Modifier les informations d'un utilisateur

L'administrateur met à jour les informations d'un utilisateur existant (rôle, coordonnées, statut). Les modifications sont immédiatement prises en compte par le système. -UC3 : Créer une équipe

L'administrateur crée une nouvelle équipe en renseignant son nom et sa description. L'équipe devient disponible pour la gestion des projets.

-UC4 : Ajouter un utilisateur à une équipe

L'administrateur affecte un ou plusieurs utilisateurs à une équipe donnée. La date d'affectation est enregistrée afin de tracer l'appartenance à l'équipe.

-UC5 : Gérer les droits et accès

L'administrateur contrôle les droits d'accès des utilisateurs en fonction de leur rôle et de leur appartenance aux équipes. Il peut intervenir en cas de problème d'accès ou d'organisation.

## **Acteur : Chef de projet**

- UC6 : Créer un projet

Le chef de projet crée un nouveau projet en renseignant le titre, la description, le budget, les dates prévues, la priorité et le statut. Le projet est automatiquement rattaché à une équipe.

- UC7 : Modifier un projet

Le chef de projet met à jour les informations d'un projet existant (statut, dates, budget ou priorité). Les changements sont visibles par les membres autorisés.

- UC8 : Ajouter des membres à un projet

Le chef de projet associe un ou plusieurs utilisateurs à un projet afin de constituer l'équipe projet. Les utilisateurs ajoutés peuvent consulter et intervenir sur le projet.

- UC9 : Créer une tâche

Le chef de projet crée une tâche liée à un projet en renseignant son titre, sa description, ses dates prévues et sa priorité. La tâche est enregistrée avec un statut initial.

- UC10 : Assigner une tâche

Le chef de projet assigne une tâche à un ou plusieurs utilisateurs. Une tâche peut également rester non assignée lorsqu'elle est placée en backlog.

- UC11 : Modifier le statut d'une tâche

Le chef de projet modifie le statut d'une tâche (à faire, en cours, bloquée, terminée). Cette action génère automatiquement une activité.

- UC12 : Suivre l'avancement du projet

Le chef de projet consulte l'avancement global du projet à travers les tâches et leurs pourcentages d'avancement. Il peut ajuster les priorités si nécessaire.

## Membre d'équipe

- UC13 : Consulter les projets

Le membre d'équipe consulte la liste des projets auxquels il est associé. Il accède aux informations générales et à l'état d'avancement.

- UC14 : Consulter les tâches

Le membre d'équipe consulte les tâches qui lui sont assignées ainsi que celles du projet. Il peut visualiser leur statut et leurs échéances.

- UC15 : Mettre à jour l'avancement d'une tâche

Le membre d'équipe met à jour le pourcentage d'avancement et le statut d'une tâche. Cette mise à jour est enregistrée et génère une activité.

- UC16 : Ajouter un commentaire

Le membre d'équipe ajoute un commentaire sur une tâche afin d'échanger avec les autres membres. Le commentaire est enregistré dans l'historique des activités.

- UC17 : Ajouter un document

Un utilisateur ajoute un document à un projet ou à une tâche. Le document est enregistré avec son auteur et sa date d'upload.

## Use cases transverses (système)

- UC18 : Génération d'une activité (log)

Le système génère automatiquement une activité lors de toute action importante sur une tâche (commentaire, changement de statut, ajout de document). Chaque activité est horodatée et associée à un utilisateur.

- UC19 : Envoi d'une notification

Le système envoie une notification à un utilisateur lors d'un événement important (création de tâche, commentaire, changement de statut). La notification peut être marquée comme lue ou non lue.

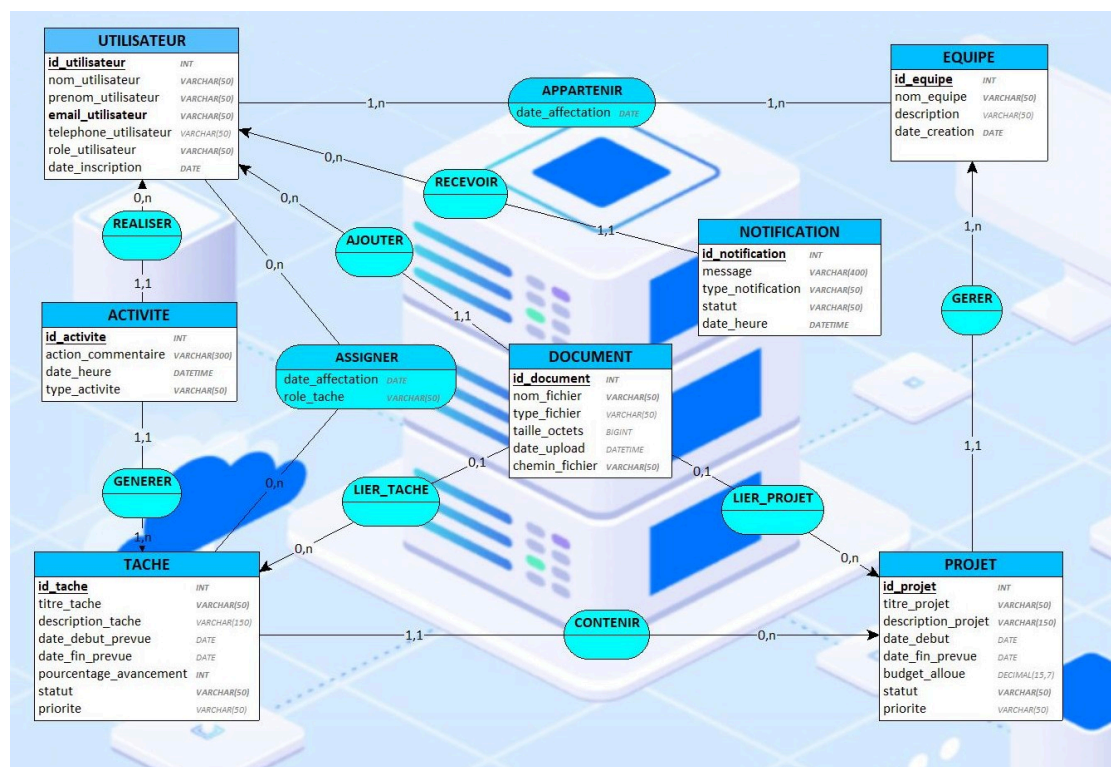
- UC20 : Consultation de l'historique

Un utilisateur consulte l'historique des activités d'une tâche afin de suivre les actions réalisées et les échanges associés.

## Modèle Conceptuel de Données (MCD)

Le **MCD (Modèle Conceptuel de Données)** est la représentation **métier** du système : il décrit *ce qu'on veut gérer* (entités comme Utilisateur, Projet, Tâche, Document...) et *comment ces éléments sont liés* (associations) avec leurs **cardinalités** (1..N, 0..N, etc.). À ce stade, on ne parle pas de SQL : l'objectif est de valider que le modèle colle à la réalité du besoin.

L'image suivante présente notre MCD dans le cadre de ce projet.

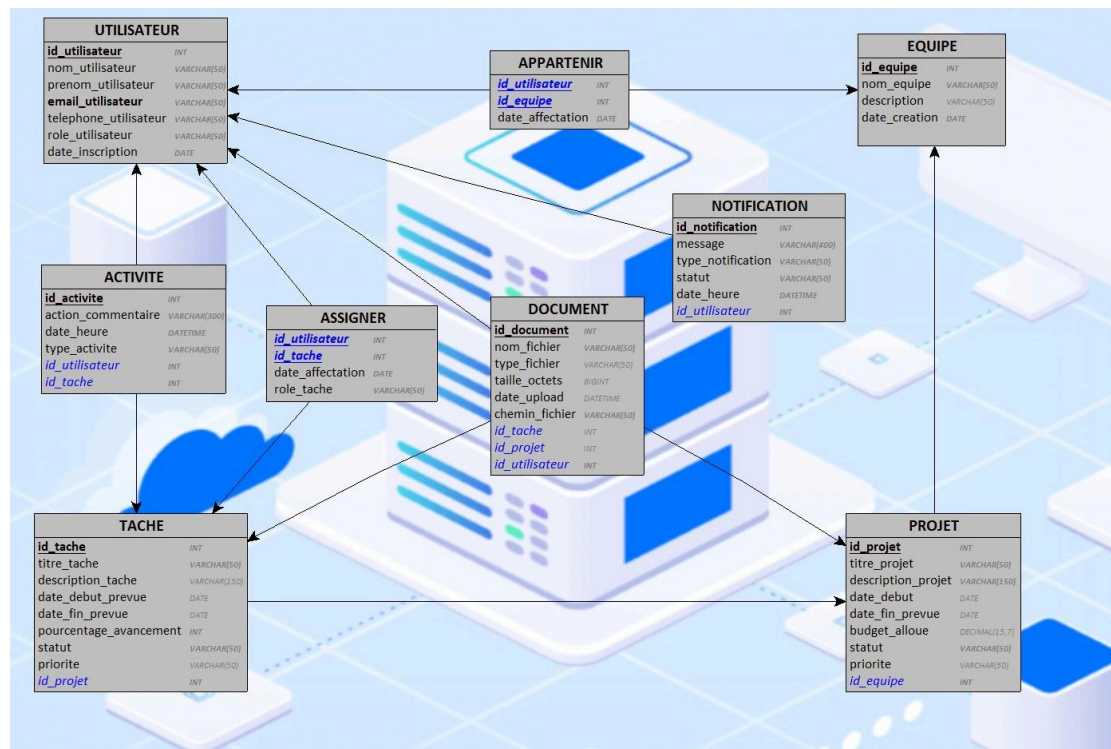


## Modèle Logique de Données (MLD)



Le **MLD (Modèle Logique de Données)** est la **traduction relationnelle** du MCD : on transforme les entités/associations en **tables**, on fixe les **clés primaires (PK)**, les **clés étrangères (FK)**, et on crée les **tables d'association** pour les relations N-N (ex. ASSIGNER pour "utilisateur ↔ tâche"). Le MLD explique donc *comment* le modèle métier se mappe dans une base relationnelle, mais reste encore relativement indépendant d'un SGBD particulier.

## Schéma du MLD



## MLD sous forme textuelle

### UTILISATEUR

UTILISATEUR = ( id\_utilisateur, nom\_utilisateur, prenom\_utilisateur, email\_utilisateur, telephone\_utilisateur, role\_utilisateur, date\_inscription );

### EQUIPE

EQUIPE = ( id\_equipe, nom\_equipe, description, date\_creation );

### PROJET

PROJET = ( id\_projet, titre\_projet, description\_projet, date\_debut, date\_fin\_prevue, budget\_alloue, statut, priorite, id\_equipe );

### TACHE

TACHE = ( id\_tache, titre\_tache, description\_tache, date\_debut\_prevue, date\_fin\_prevue, pourcentage\_avancement, statut, priorite, id\_projet );

## ACTIVITE

ACTIVITE = ( id\_activite, action\_commentaire, date\_heure, type\_activite, id\_utilisateur, id\_tache );

## DOCUMENT

DOCUMENT = ( id\_document, nom\_fichier, type\_fichier, taille\_octets, date\_upload, chemin\_fichier, id\_tache, id\_projet, id\_utilisateur );

## NOTIFICATION

NOTIFICATION = ( id\_notification, message, type\_notification, statut, date\_heure, id\_utilisateur );

## APPARTENIR (association UTILISATEUR ↔ EQUIPE devenue table)

APPARTENIR = ( id\_utilisateur, id\_equipe, date\_affectation );

## ASSIGNER (association UTILISATEUR ↔ TACHE devenue table)

ASSIGNER = ( id\_utilisateur, id\_tache, date\_affectation, role\_tache );

# Modèle Physique de Données (MPD)

Dans cete section, nous présentons le Modèle Physique de Données (MPD) en tableaux (colonnes, types, clés, contraintes).

Le **MPD (Modèle Physique de Données)** est la version **technique prête à être implémentée** dans un SGBD précis (MySQL, PostgreSQL...). Il reprend le MLD et ajoute tout ce qui est nécessaire pour créer réellement la base : **types SQL** ( INT , VARCHAR , DATE , DATETIME ...), **contraintes** ( NOT NULL , UNIQUE , CHECK ), règles de cohérence (ex. avancement entre 0 et 100), éventuellement **index** et conventions de stockage. Autrement dit, le MPD dit exactement "comment la base sera stockée" dans un environnement réel.

## Table ACTIVITE

| table    | column             | type         | nullable | key | reference | cc |
|----------|--------------------|--------------|----------|-----|-----------|----|
| ACTIVITE | action_commentaire | VARCHAR(300) | Non      |     |           |    |

| table    | column         | type        | nullable | key | reference                   | constraints                                                    |
|----------|----------------|-------------|----------|-----|-----------------------------|----------------------------------------------------------------|
| ACTIVITE | date_heure     | DATETIME    | Non      |     |                             |                                                                |
| ACTIVITE | type_activite  | VARCHAR(50) | Non      |     |                             | Clé étrangère vers le type_activite de la table ACTIVITE ('C') |
| ACTIVITE | id_tache       | INT         | Non      | FK  | TACHE(id_tache)             |                                                                |
| ACTIVITE | id_utilisateur | INT         | Non      | FK  | UTILISATEUR(id_utilisateur) |                                                                |
| ACTIVITE | id_activite    | INT         | Non      | PK  |                             | PK                                                             |

--

## Table APPARTENIR

| table      | column           | type | nullable | key | reference                   | constraints |
|------------|------------------|------|----------|-----|-----------------------------|-------------|
| APPARTENIR | date_affectation | DATE | Non      |     |                             |             |
| APPARTENIR | id_equipe        | INT  | Non      | PK  | EQUIPE(id_equipe)           | PRIMARY KEY |
| APPARTENIR | id_utilisateur   | INT  | Non      | PK  | UTILISATEUR(id_utilisateur) | PRIMARY KEY |

--

## Table ASSIGNER

| table    | column           | type        | nullable | key | reference                   | constraints |
|----------|------------------|-------------|----------|-----|-----------------------------|-------------|
| ASSIGNER | date_affectation | DATE        | Non      |     |                             |             |
| ASSIGNER | role_tache       | VARCHAR(50) | Non      |     |                             |             |
| ASSIGNER | id_tache         | INT         | Oui      | PK  | TACHE(id_tache)             | PRIMARY KEY |
| ASSIGNER | id_utilisateur   | INT         | Oui      | PK  | UTILISATEUR(id_utilisateur) | PRIMARY KEY |

--

## Table DOCUMENT

| table    | column        | type | nullable | key | reference | constraints                                                                         |
|----------|---------------|------|----------|-----|-----------|-------------------------------------------------------------------------------------|
| DOCUMENT | (Règle table) |      |          |     |           | CHECK (id_tache IS NOT NULL AND id_utilisateur IS NOT NULL) lié à la table ACTIVITE |

| table    | column         | type        | nullable | key | reference                   | constraints                      |
|----------|----------------|-------------|----------|-----|-----------------------------|----------------------------------|
|          |                |             |          |     |                             | projets<br>une tâche             |
| DOCUMENT | chemin_fichier | VARCHAR(50) | Non      |     |                             |                                  |
| DOCUMENT | date_upload    | DATETIME    | Oui      |     |                             |                                  |
| DOCUMENT | nom_fichier    | VARCHAR(50) | Non      |     |                             |                                  |
| DOCUMENT | taille_octets  | BIGINT      | Oui      |     |                             | (taille<br>fichier<br>en octets) |
| DOCUMENT | type_fichier   | VARCHAR(50) | Oui      |     |                             |                                  |
| DOCUMENT | id_projet      | INT         | Oui      | FK  | PROJET(id_projet)           |                                  |
| DOCUMENT | id_tache       | INT         | Oui      | FK  | TACHE(id_tache)             |                                  |
| DOCUMENT | id_utilisateur | INT         | Non      | FK  | UTILISATEUR(id_utilisateur) |                                  |
| DOCUMENT | id_document    | INT         | Non      | PK  |                             | PRIMARY<br>KEY                   |

--

## Table EQUIPE

| table  | column        | type        | nullable | key | reference | constraints |
|--------|---------------|-------------|----------|-----|-----------|-------------|
| EQUIPE | date_creation | DATE        | Non      |     |           |             |
| EQUIPE | description   | VARCHAR(50) | Oui      |     |           |             |
| EQUIPE | nom_equipe    | VARCHAR(50) | Non      |     |           |             |
| EQUIPE | id_equipe     | INT         | Non      | PK  |           | PRIMARY KEY |

--

## Table NOTIFICATION

| table        | column            | type         | nullable | key | reference                   | constraints |
|--------------|-------------------|--------------|----------|-----|-----------------------------|-------------|
| NOTIFICATION | date_heure        | DATETIME     | Non      |     |                             |             |
| NOTIFICATION | message           | VARCHAR(400) | Non      |     |                             |             |
| NOTIFICATION | statut            | VARCHAR(50)  | Non      |     |                             |             |
| NOTIFICATION | type_notification | VARCHAR(50)  | Non      |     |                             |             |
| NOTIFICATION | id_utilisateur    | INT          | Non      | FK  | UTILISATEUR(id_utilisateur) |             |
| NOTIFICATION | id_notification   | INT          | Oui      | PK  |                             |             |

--

## Table PROJET

| table  | column             | type          | nullable | key | reference         | constraints            |
|--------|--------------------|---------------|----------|-----|-------------------|------------------------|
| PROJET | budget_alloue      | DECIMAL(15,7) | Oui      |     |                   | CHECK >= 0             |
| PROJET | date_debut         | DATE          | Oui      |     |                   |                        |
| PROJET | date_fin_prevue    | DATE          | Oui      |     |                   | CHECK >= da            |
| PROJET | description_projet | VARCHAR(150)  | Non      |     |                   |                        |
| PROJET | priorite           | VARCHAR(50)   | Oui      |     |                   | CHECK IN ('H/          |
| PROJET | statut             | VARCHAR(50)   | Non      |     |                   | CHECK IN ('EN_PLANIFIC |
| PROJET | titre_projet       | VARCHAR(50)   | Non      |     |                   |                        |
| PROJET | id_equipe          | INT           | Non      | FK  | EQUIPE(id_equipe) |                        |
| PROJET | id_projet          | INT           | Non      | PK  |                   | PRIMARY KEY            |

--

## Table TACHE

| table | column                 | type         | nullable | key | reference         | constra          |
|-------|------------------------|--------------|----------|-----|-------------------|------------------|
| TACHE | date_debut_prevue      | DATE         | Oui      |     |                   |                  |
| TACHE | date_fin_prevue        | DATE         | Oui      |     |                   |                  |
| TACHE | description_tache      | VARCHAR(150) | Oui      |     |                   |                  |
| TACHE | pourcentage_avancement | INT          | Non      |     |                   | CHECK I          |
| TACHE | priorite               | VARCHAR(50)  | Oui      |     |                   | CHECK I          |
| TACHE | statut                 | VARCHAR(20)  | Non      |     |                   | CHECK I ('A_FAIR |
| TACHE | titre_tache            | VARCHAR(50)  | Non      |     |                   |                  |
| TACHE | id_projet              | INT          | Non      | FK  | PROJET(id_projet) |                  |
| TACHE | id_tache               | INT          | Non      | PK  |                   | PRIMAR           |

--

## Table UTILISATEUR

| table       | column                | type        | nullable | key | reference | constraints             |
|-------------|-----------------------|-------------|----------|-----|-----------|-------------------------|
| UTILISATEUR | date_inscription      | DATE        | Non      |     |           |                         |
| UTILISATEUR | email_utilisateur     | VARCHAR(50) | Non      |     |           | UNIQUE                  |
| UTILISATEUR | nom_utilisateur       | VARCHAR(50) | Non      |     |           |                         |
| UTILISATEUR | prenom_utilisateur    | VARCHAR(50) | Non      |     |           |                         |
| UTILISATEUR | role_utilisateur      | VARCHAR(50) | Non      |     |           | CHECK IN ('ADMIN','CHEF |
| UTILISATEUR | telephone_utilisateur | VARCHAR(50) | Non      |     |           |                         |
| UTILISATEUR | id_utilisateur        | INT         | Non      | PK  |           | PRIMARY KEY             |

## Dictionnaire des données

Le **dictionnaire des données** est un **document explicatif** (souvent un tableau Word/Excel) qui décrit chaque champ du MPD : sa signification métier, son type, s'il est obligatoire, ses valeurs possibles (domaines/énumérations), ses contraintes, ses liens (FK), et parfois des exemples. Il ne remplace pas les modèles : il sert à rendre le schéma **compréhensible et répliquable** par d'autres (développeurs, analystes, nouveaux membres de l'équipe).

### Entité UTILISATEUR

**Description :** Personne utilisant la plateforme (admin, chef de projet ou membre).

**Règles & contraintes clés :**

- **PK :** id\_utilisateur (AUTO\_INCREMENT)
- **UNIQUE :** email\_utilisateur
- **CHECK :** role\_utilisateur ∈ { ADMIN , CHEF\_PROJET , MEMBRE }

| Attribut              | Description                          | Type         | Domaine       | Contraintes                 | Clé (PK/F) |
|-----------------------|--------------------------------------|--------------|---------------|-----------------------------|------------|
| id_utilisateur        | Identifiant unique de l'utilisateur. | INT          | Entier        | PRIMARY KEY, AUTO_INCREMENT | PK         |
| nom_utilisateur       | Nom de famille.                      | VARCHAR(50)  | Texte (≤ 50)  | NOT NULL                    |            |
| prenom_utilisateur    | Prénom.                              | VARCHAR(50)  | Texte (≤ 50)  | NOT NULL                    |            |
| email_utilisateur     | Email (unique).                      | VARCHAR(255) | Email (≤ 255) | NOT NULL, UNIQUE            |            |
| telephone_utilisateur | Téléphone.                           | VARCHAR(50)  | Texte (≤ 50)  | NULL autorisé               |            |
| role_utilisateur      | Rôle applicatif.                     | VARCHAR(20)  | Enum: ADMIN   | NOT NULL, CHECK             |            |

| Attribut         | Description         | Type | Domaine                 | Contraintes | Clé (PK/F |
|------------------|---------------------|------|-------------------------|-------------|-----------|
|                  |                     |      | CHEF_PROJET<br>  MEMBRE |             |           |
| date_inscription | Date d'inscription. | DATE | Date                    | NOT NULL    |           |
| --               |                     |      |                         |             |           |

## Entité EQUIPE

**Description :** Regroupe des utilisateurs dans une équipe de travail.

**Règles & contraintes clés :**

- **PK :** id\_equipe (AUTO\_INCREMENT)
- **UNIQUE :** nom\_equipe

| Attribut      | Description                     | Type         | Domaine       | Contraintes                 | Clé (PK/FK) | Exempl                  |
|---------------|---------------------------------|--------------|---------------|-----------------------------|-------------|-------------------------|
| id_equipe     | Identifiant unique de l'équipe. | INT          | Entier        | PRIMARY KEY, AUTO_INCREMENT | PK          | 10                      |
| nom_equipe    | Nom de l'équipe.                | VARCHAR(80)  | Texte (≤ 80)  | NOT NULL, UNIQUE            |             | Program Santé & Terrain |
| description   | Description de l'équipe.        | VARCHAR(255) | Texte (≤ 255) | NULL autorisé               |             | Équipe suivi ter        |
| date_creation | Date de création.               | DATE         | Date          | NOT NULL                    |             | 2026-0                  |

--

## Table d'association APPARTENIR (Utilisateur ↔ Équipe)

**Description :** Association **N–N** entre utilisateurs et équipes, avec date d'affectation.

**Règles & contraintes clés :**

- **PK composite :** ( id\_utilisateur , id\_equipe )
- **FK :** id\_utilisateur → UTILISATEUR(id\_utilisateur) ; id\_equipe → EQUIPE(id\_equipe)

| Attribut       | Description           | Type | Domaine | Contraintes | Clé (PK/FK)                          | Ex |
|----------------|-----------------------|------|---------|-------------|--------------------------------------|----|
| id_utilisateur | Utilisateur membre de | INT  | Entier  | NOT NULL    | PK, FK → UTILISATEUR(id_utilisateur) | 1  |

| Attribut         | Description         | Type | Domaine | Contraintes | Clé (PK/FK)                | Ex   |
|------------------|---------------------|------|---------|-------------|----------------------------|------|
|                  | l'équipe.           |      |         |             |                            |      |
| id_equipe        | Équipe concernée.   | INT  | Entier  | NOT NULL    | PK, FK → EQUIPE(id_equipe) | 10   |
| date_affectation | Date d'affectation. | DATE | Date    | NOT NULL    |                            | 2001 |

--

## Entité PROJET

**Description :** Projet porté par une équipe (planning, budget, statut, priorité).

**Règles & contraintes clés :**

- **PK :** id\_projet (AUTO\_INCREMENT)
- **FK :** id\_equipe → EQUIPE(id\_equipe)
- **DEFAULT :** budget\_alloue = 0
- **CHECK :** budget\_alloue >= 0
- **CHECK :** date\_fin\_prevue >= date\_debut
- **CHECK :** statut ∈ { EN\_PLANIFICATION , EN\_COURS , EN\_PAUSE , TERMINE }
- **CHECK :** priorite ∈ { HAUTE , MOYENNE , BASSE }

| Attribut           | Description                   | Type          | Domaine        | Contraintes                      | Clé (PK/FK) |
|--------------------|-------------------------------|---------------|----------------|----------------------------------|-------------|
| id_projet          | Identifiant unique du projet. | INT           | Entier         | PRIMARY KEY, AUTO_INCREMENT      | PK          |
| titre_projet       | Titre du projet.              | VARCHAR(120)  | Texte (≤ 120)  | NOT NULL                         |             |
| description_projet | Description détaillée.        | VARCHAR(500)  | Texte (≤ 500)  | NOT NULL                         |             |
| date_debut         | Date de démarrage.            | DATE          | Date           | NOT NULL                         |             |
| date_fin_prevue    | Date de fin prévue.           | DATE          | Date           | NOT NULL, CHECK (>= date_debut)  |             |
| budget_alloue      | Budget alloué.                | DECIMAL(15,2) | Montant        | NOT NULL, DEFAULT 0, CHECK (>=0) |             |
| statut             | Statut du projet.             | VARCHAR(20)   | Enum autorisée | NOT NULL, CHECK                  |             |
| priorite           | Priorité du projet.           | VARCHAR(10)   | Enum autorisée | NOT NULL, CHECK                  |             |



| Attribut  | Description      | Type | Domaine | Contraintes | Clé (PK/FI        |
|-----------|------------------|------|---------|-------------|-------------------|
| id_equipe | Équipe porteuse. | INT  | Entier  | NOT NULL    | FK → EQUIPE(id_eq |

--

## Entité TACHE

**Description :** Tâche rattachée à un projet (statut, priorité, avancement).

**Règles & contraintes clés :**

- **PK :** id\_tache (AUTO\_INCREMENT)
- **FK :** id\_projet → PROJET(id\_projet)
- **DEFAULT :** pourcentage\_avancement = 0
- **CHECK :** pourcentage\_avancement entre 0 et 100
- **CHECK :** cohérence des dates (si les deux dates existent, fin ≥ début)
- **CHECK :** statut ∈ { A\_FAIRE , EN\_COURS , TERMINE , BLOQUE }
- **CHECK :** priorite ∈ { HAUTE , MOYENNE , BASSE }

| Attribut               | Description                     | Type         | Domaine        | Contraintes                                                 | Clé       |
|------------------------|---------------------------------|--------------|----------------|-------------------------------------------------------------|-----------|
| id_tache               | Identifiant unique de la tâche. | INT          | Entier         | PRIMARY KEY, AUTO_INCREMENT                                 | PK        |
| titre_tache            | Titre de la tâche.              | VARCHAR(120) | Texte (≤ 120)  | NOT NULL                                                    |           |
| description_tache      | Description de la tâche.        | VARCHAR(500) | Texte (≤ 500)  | NULL autorisé                                               |           |
| date_debut_prevue      | Début prévu.                    | DATE         | Date           | NULL autorisé                                               |           |
| date_fin_prevue        | Fin prévue.                     | DATE         | Date           | NULL autorisé, CHECK (si début renseigné alors fin ≥ début) |           |
| pourcentage_avancement | Avancement (%)                  | INT          | Entier         | NOT NULL, DEFAULT 0, CHECK (0..100)                         |           |
| statut                 | Statut de la tâche.             | VARCHAR(20)  | Enum autorisée | NOT NULL, CHECK                                             |           |
| priorite               | Priorité de la tâche.           | VARCHAR(10)  | Enum autorisée | NOT NULL, CHECK                                             |           |
| id_projet              | Projet parent.                  | INT          | Entier         | NOT NULL                                                    | FK → PROJ |

--

## Table d'association **ASSIGNER** (Utilisateur ↔ Tâche)

**Description :** Association **N–N** entre utilisateurs et tâches, avec date d'affectation et rôle dans la tâche.

### Règles & contraintes clés :

- **PK composite :** ( id\_utilisateur , id\_tache )
- **FK :** id\_utilisateur → UTILISATEUR(id\_utilisateur) ; id\_tache → TACHE(id\_tache)

| Attribut         | Description          | Type        | Domaine      | Contraintes | Clé (PK/FK)                     |
|------------------|----------------------|-------------|--------------|-------------|---------------------------------|
| id_utilisateur   | Utilisateur assigné. | INT         | Entier       | NOT NULL    | PK, FK → UTILISATEUR(id_utilisa |
| id_tache         | Tâche concernée.     | INT         | Entier       | NOT NULL    | PK, FK → TACHE(id_tac           |
| date_affectation | Date d'assignation.  | DATE        | Date         | NOT NULL    |                                 |
| role_tache       | Rôle dans la tâche.  | VARCHAR(50) | Texte (≤ 50) | NOT NULL    |                                 |

--

## Entité **ACTIVITE**

**Description :** Journal (logs) des actions réalisées sur les tâches.

### Règles & contraintes clés :

- **PK :** id\_activite (AUTO\_INCREMENT)
- **FK :** id\_utilisateur → UTILISATEUR(id\_utilisateur) ; id\_tache → TACHE(id\_tache)
- **CHECK :** type\_activite ∈ { COMMENTAIRE , MAJ\_STATUT , MAJ\_AVANCEMENT , AJOUT\_DOCUMENT , REASSIGNATION

| Attribut    | Description                       | Type         | Domaine       | Contraintes                 | Clé (PK/FI |
|-------------|-----------------------------------|--------------|---------------|-----------------------------|------------|
| id_activite | Identifiant unique de l'activité. | INT          | Entier        | PRIMARY KEY, AUTO_INCREMENT | PK         |
| contenu     | Commentaire / action (texte).     | VARCHAR(300) | Texte (≤ 300) | NULL autorisé               |            |

| Attribut       | Description      | Type        | Domaine        | Contraintes     | Clé (PK/FK)           |
|----------------|------------------|-------------|----------------|-----------------|-----------------------|
| date_heure     | Date et heure.   | DATETIME    | Date & heure   | NOT NULL        |                       |
| type_activite  | Type d'action.   | VARCHAR(30) | Enum autorisée | NOT NULL, CHECK |                       |
| id_utilisateur | Auteur.          | INT         | Entier         | NOT NULL        | FK → UTILISATEUR(id_u |
| id_tache       | Tâche concernée. | INT         | Entier         | NOT NULL        | FK → TACHE(id_ta      |

--

## Entité DOCUMENT

**Description :** Document téléversé, rattaché à **une tâche OU un projet** (règle XOR), et à un utilisateur auteur.

**Règles & contraintes clés :**

- **PK :** id\_document (AUTO\_INCREMENT)
- **FK :** id\_tache → TACHE(id\_tache) ; id\_projet → PROJET(id\_projet) ; id\_utilisateur → UTILISATEUR(id\_utilisateur)
- **CHECK (XOR) :** un document est lié **soit** à une tâche **soit** à un projet (pas les deux) :  
CHECK ( (id\_tache IS NULL) <> (id\_projet IS NULL) )

| Attribut       | Description                     | Type         | Domaine       | Contraintes                 | Clé (P            |
|----------------|---------------------------------|--------------|---------------|-----------------------------|-------------------|
| id_document    | Identifiant unique du document. | INT          | Entier        | PRIMARY KEY, AUTO_INCREMENT | PK                |
| nom_fichier    | Nom du fichier.                 | VARCHAR(255) | Texte (≤ 255) | NOT NULL                    |                   |
| type_fichier   | Type/extension.                 | VARCHAR(80)  | Texte (≤ 80)  | NULL autorisé               |                   |
| taille_octets  | Taille du fichier en octets.    | BIGINT       | Entier        | NULL autorisé               |                   |
| date_upload    | Date et heure d'upload.         | DATETIME     | Date & heure  | NOT NULL                    |                   |
| chemin_fichier | Chemin/URI de stockage.         | VARCHAR(500) | Texte (≤ 500) | NOT NULL                    |                   |
| id_tache       | Tâche rattachée (optionnel).    | INT          | Entier        | NULL autorisé               | FK → TACHE(i      |
| id_projet      | Projet rattaché (optionnel).    | INT          | Entier        | NULL autorisé               | FK → PROJET(      |
| id_utilisateur | Auteur du téléversement.        | INT          | Entier        | NOT NULL                    | FK → UTILISATEUR( |

## Entité NOTIFICATION

**Description :** Notification envoyée à un utilisateur (assignation, changement de statut, etc.).

### Règles & contraintes clés :

- **PK :** `id_notification` (AUTO\_INCREMENT)
- **FK :** `id_utilisateur` → `UTILISATEUR(id_utilisateur)`
- **CHECK :** `statut` ∈ { LU , NON\_LU }

| Attribut          | Description                            | Type         | Domaine           | Contraintes                 | Clé (PK)             |
|-------------------|----------------------------------------|--------------|-------------------|-----------------------------|----------------------|
| id_notification   | Identifiant unique de la notification. | INT          | Entier            | PRIMARY KEY, AUTO_INCREMENT | PK                   |
| message           | Message affiché.                       | VARCHAR(400) | Texte (≤ 400)     | NOT NULL                    |                      |
| type_notification | Type de notification.                  | VARCHAR(50)  | Texte (≤ 50)      | NOT NULL                    |                      |
| statut            | Statut de lecture.                     | VARCHAR(10)  | Enum: LU   NON_LU | NOT NULL, CHECK             |                      |
| date_heure        | Date et heure d'émission.              | DATETIME     | Date & heure      | NOT NULL                    |                      |
| id_utilisateur    | Destinataire.                          | INT          | Entier            | NOT NULL                    | FK → UTILISATEUR(id) |

## Contraintes d'intégrité

Cette partie liste les contraintes d'intégrité du MPD. Les tableaux ci-dessous sont conçus pour un affichage stable (GitHub, VS Code, Obsidian, etc.).

## Définition

Les contraintes d'intégrité sont des règles appliquées par le SGBD pour empêcher l'enregistrement de données incohérentes. Elles couvrent l'intégrité d'entité (chaque ligne est unique via une clé primaire), l'intégrité référentielle (les clés étrangères ne peuvent pointer que vers des enregistrements existants), l'intégrité de domaine (valeurs obligatoires, unicité, plages/ensembles autorisés) et l'intégrité métier (règles propres au contexte, comme la cohérence des dates ou l'exclusivité « projet OU tâche » pour un document). En cas de violation, l'insertion ou la mise à jour est refusée.

# Tableau global des contraintes

| Table      | Type d'intégrité | Mécanisme   | Contrainte (SQL)                                                                       |
|------------|------------------|-------------|----------------------------------------------------------------------------------------|
| activite   | Domaine          | NOT NULL    | NOT NULL sur: date_heure, id_tache, id_ut                                              |
| activite   | Entité           | PRIMARY KEY | PRIMARY KEY (id_activite)                                                              |
| activite   | Domaine / Métier | CHECK       | CONSTRAINT ck_act_type CHECK (type_activi ('COMMENTAIRE', 'MAJ_STATUT', 'MAJ_AVANCEMEN |
| activite   | Référentielle    | FOREIGN KEY | CONSTRAINT fk_act_user FOREIGN KEY (id_ut utilisateur(id_utilisateur)                  |
| activite   | Référentielle    | FOREIGN KEY | CONSTRAINT fk_act_task FOREIGN KEY (id_ta                                              |
| appartenir | Domaine          | NOT NULL    | NOT NULL sur: date_affectation, id_equipe                                              |
| appartenir | Entité           | PRIMARY KEY | PRIMARY KEY (id_utilisateur, id_equipe)                                                |
| appartenir | Référentielle    | FOREIGN KEY | CONSTRAINT fk_app_user FOREIGN KEY (id_ut utilisateur(id_utilisateur)                  |
| appartenir | Référentielle    | FOREIGN KEY | CONSTRAINT fk_app_team FOREIGN KEY (id_eq                                              |
| assigner   | Domaine          | NOT NULL    | NOT NULL sur: date_affectation, id_tache,                                              |
| assigner   | Entité           | PRIMARY KEY | PRIMARY KEY (id_utilisateur, id_tache)                                                 |
| assigner   | Référentielle    | FOREIGN KEY | CONSTRAINT fk_assign_user FOREIGN KEY (id utilisateur(id_utilisateur)                  |

| Table        | Type d'intégrité | Mécanisme   | Contrainte (SQL)                                                         |
|--------------|------------------|-------------|--------------------------------------------------------------------------|
| assigner     | Référentielle    | FOREIGN KEY | CONSTRAINT fk_assign_task FOREIGN KEY (id                                |
| document     | Domaine          | NOT NULL    | NOT NULL sur: chemin_fichier, date_upload                                |
| document     | Entité           | PRIMARY KEY | PRIMARY KEY (id_document)                                                |
| document     | Domaine / Métier | CHECK       | CONSTRAINT ck_doc_xor CHECK ((id_tache IS                                |
| document     | Référentielle    | FOREIGN KEY | CONSTRAINT fk_doc_task FOREIGN KEY (id_ta                                |
| document     | Référentielle    | FOREIGN KEY | CONSTRAINT fk_doc_projet FOREIGN KEY (id_                                |
| document     | Référentielle    | FOREIGN KEY | CONSTRAINT fk_doc_user FOREIGN KEY (id_ut<br>utilisateur(id_utilisateur) |
| equipe       | Domaine          | NOT NULL    | NOT NULL sur: date_creation, nom_equipe                                  |
| equipe       | Entité           | PRIMARY KEY | PRIMARY KEY (id_equipe)                                                  |
| equipe       | Domaine          | UNIQUE      | UNIQUE (nom_equipe)                                                      |
| notification | Domaine          | NOT NULL    | NOT NULL sur: date_heure, id_utilisateur,                                |
| notification | Entité           | PRIMARY KEY | PRIMARY KEY (id_notification)                                            |
| notification | Domaine / Métier | CHECK       | CONSTRAINT ck_notif_statut CHECK (statut                                 |
| notification | Référentielle    | FOREIGN KEY | CONSTRAINT fk_notif_user FOREIGN KEY (id_<br>utilisateur(id_utilisateur) |

| Table         | Type d'intégrité | Mécanisme   | Contrainte (SQL)                                                                     |
|---------------|------------------|-------------|--------------------------------------------------------------------------------------|
| projet        | Domaine          | NOT NULL    | NOT NULL sur: budget_alloue, date_debut, id_equipe, priorite, statut, titre_projet   |
| projet        | Entité           | PRIMARY KEY | PRIMARY KEY (id_projet)                                                              |
| projet        | Domaine          | DEFAULT     | DEFAULT 0 sur budget_alloue                                                          |
| projet        | Domaine / Métier | CHECK       | CONSTRAINT ck_projet_budget CHECK (budget                                            |
| projet        | Domaine / Métier | CHECK       | CONSTRAINT ck_projet_dates CHECK (date_fi                                            |
| projet        | Domaine / Métier | CHECK       | CONSTRAINT ck_projet_statut CHECK (statut ('EN_PLANIFICATION','EN_COURS','EN_PAUSE', |
| projet        | Domaine / Métier | CHECK       | CONSTRAINT ck_projet_priorite CHECK (prio                                            |
| projet        | Référentielle    | FOREIGN KEY | CONSTRAINT fk_projet_equipe FOREIGN KEY (equipe(id_equipe)                           |
| projet_membre | Domaine          | NOT NULL    | NOT NULL sur: date_affectation, id_projet                                            |
| projet_membre | Entité           | PRIMARY KEY | PRIMARY KEY (id_projet, id_utilisateur)                                              |
| projet_membre | Domaine / Métier | CHECK       | CONSTRAINT ck_pm_role CHECK (role_projet                                             |
| projet_membre | Référentielle    | FOREIGN KEY | CONSTRAINT fk_pm_projet FOREIGN KEY (id_p                                            |
| projet_membre | Référentielle    | FOREIGN KEY | CONSTRAINT fk_pm_user FOREIGN KEY (id_utilisateur(id_utilisateur)                    |
| tache         | Domaine          | NOT NULL    | NOT NULL sur: id_projet, pourcentage_avan                                            |
| tache         | Entité           | PRIMARY KEY | PRIMARY KEY (id_tache)                                                               |
| tache         | Domaine          | DEFAULT     | DEFAULT 0 sur pourcentage_avancement                                                 |
| tache         | Domaine / Métier | CHECK       | CONSTRAINT ck_tache_avancement CHECK (pou 100)                                       |

| Table       | Type d'intégrité | Mécanisme   | Contrainte (SQL)                                                                     |
|-------------|------------------|-------------|--------------------------------------------------------------------------------------|
| tache       | Domaine / Métier | CHECK       | CONSTRAINT ck_tache_dates CHECK (date_fin IS NULL OR date_fin_prevue >= date_debut_p |
| tache       | Domaine / Métier | CHECK       | CONSTRAINT ck_tache_statut CHECK (statut ('A_FAIRE','EN_COURS','TERMINE','BLOQUE'))  |
| tache       | Domaine / Métier | CHECK       | CONSTRAINT ck_tache_priorite CHECK (prior                                            |
| tache       | Référentielle    | FOREIGN KEY | CONSTRAINT fk_tache_projet FOREIGN KEY (i projet(id_projet)                          |
| utilisateur | Domaine          | NOT NULL    | NOT NULL sur: date_inscription, email_utili prenom_utilisateur, role_utilisateur     |
| utilisateur | Entité           | PRIMARY KEY | PRIMARY KEY (id_utilisateur)                                                         |
| utilisateur | Domaine          | UNIQUE      | UNIQUE (email_utilisateur)                                                           |
| utilisateur | Domaine / Métier | CHECK       | CONSTRAINT ck_user_role CHECK (role_utili ('ADMIN','CHEF_PROJET','MEMBRE'))          |

## Détail par table

### ACTIVITE

| Type d'intégrité | Mécanisme   | Contrainte                                                                                              |
|------------------|-------------|---------------------------------------------------------------------------------------------------------|
| Domaine          | NOT NULL    | NOT NULL sur: date_heure, id_tache, id_utilisateur, typ                                                 |
| Entité           | PRIMARY KEY | PRIMARY KEY (id_activite)                                                                               |
| Domaine / Métier | CHECK       | CONSTRAINT ck_act_type CHECK (type_activite IN ('COMMENTAIRE','MAJ_STATUT','MAJ_AVANCEMENT','AJOUT_DOCU |
| Référentielle    | FOREIGN KEY | CONSTRAINT fk_act_user FOREIGN KEY (id_utilisateur) REF utilisateur(id_utilisateur)                     |
| Référentielle    | FOREIGN KEY | CONSTRAINT fk_act_task FOREIGN KEY (id_tache) REFERENCE                                                 |



| Type d'intégrité | Mécanisme | Contrainte |
|------------------|-----------|------------|
|------------------|-----------|------------|

## APPARTENIR

| Type d'intégrité | Mécanisme   | Contrainte                                                                                          | Sens                                                                                                                    |
|------------------|-------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Domaine          | NOT NULL    | NOT NULL sur:<br>date_affectation, id_equipe,<br>id_utilisateur                                     | Champs obligatoires (valeur requise).                                                                                   |
| Entité           | PRIMARY KEY | PRIMARY KEY (id_utilisateur,<br>id_equipe)                                                          | Identifie de façon unique chaque enregistrement.                                                                        |
| Référentielle    | FOREIGN KEY | CONSTRAINT fk_app_user<br>FOREIGN KEY (id_utilisateur)<br>REFERENCES<br>utilisateur(id_utilisateur) | Empêche appartenir.id_utilisateur de référencer une valeur inexistante (doit exister dans utilisateur(id_utilisateur)). |
| Référentielle    | FOREIGN KEY | CONSTRAINT fk_app_team<br>FOREIGN KEY (id_equipe)<br>REFERENCES equipe(id_equipe)                   | Empêche appartenir.id_equipe de référencer une valeur inexistante (doit exister dans equipe(id_equipe)).                |

## ASSIGNER

| Type d'intégrité | Mécanisme   | Contrainte                                                                                             | Sens                                                                                                                  |
|------------------|-------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Domaine          | NOT NULL    | NOT NULL sur:<br>date_affectation, id_tache,<br>id_utilisateur, role_tache                             | Champs obligatoires (valeur requise).                                                                                 |
| Entité           | PRIMARY KEY | PRIMARY KEY (id_utilisateur,<br>id_tache)                                                              | Identifie de façon unique chaque enregistrement.                                                                      |
| Référentielle    | FOREIGN KEY | CONSTRAINT fk_assign_user<br>FOREIGN KEY (id_utilisateur)<br>REFERENCES<br>utilisateur(id_utilisateur) | Empêche assigner.id_utilisateur de référencer une valeur inexistante (doit exister dans utilisateur(id_utilisateur)). |
| Référentielle    | FOREIGN KEY | CONSTRAINT fk_assign_task<br>FOREIGN KEY (id_tache)<br>REFERENCES tache(id_tache)                      | Empêche assigner.id_tache de référencer une valeur inexistante (doit exister dans tache(id_tache)).                   |

## DOCUMENT

| Type d'intégrité | Mécanisme   | Contrainte                                                                                 | Sens                                                                                                                  |
|------------------|-------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Domaine          | NOT NULL    | NOT NULL sur: chemin_fichier, date_upload, id_utilisateur, nom_fichier                     | Champs obligatoires (valeur requise).                                                                                 |
| Entité           | PRIMARY KEY | PRIMARY KEY (id_document)                                                                  | Identifie de façon unique chaque enregistrement.                                                                      |
| Domaine / Métier | CHECK       | CONSTRAINT ck_doc_xor CHECK ((id_tache IS NULL) <> (id_projet IS NULL))                    | Un document est rattaché à une tâche OU à un projet, pas les deux.                                                    |
| Référentielle    | FOREIGN KEY | CONSTRAINT fk_doc_task FOREIGN KEY (id_tache) REFERENCES tache(id_tache)                   | Empêche document.id_tache de référencer une valeur inexistante (doit exister dans tache(id_tache)).                   |
| Référentielle    | FOREIGN KEY | CONSTRAINT fk_doc_projet FOREIGN KEY (id_projet) REFERENCES projet(id_projet)              | Empêche document.id_projet de référencer une valeur inexistante (doit exister dans projet(id_projet)).                |
| Référentielle    | FOREIGN KEY | CONSTRAINT fk_doc_user FOREIGN KEY (id_utilisateur) REFERENCES utilisateur(id_utilisateur) | Empêche document.id_utilisateur de référencer une valeur inexistante (doit exister dans utilisateur(id_utilisateur)). |

## EQUIPE

| Type d'intégrité | Mécanisme   | Contrainte                              | Sens                                             |
|------------------|-------------|-----------------------------------------|--------------------------------------------------|
| Domaine          | NOT NULL    | NOT NULL sur: date_creation, nom_equipe | Champs obligatoires (valeur requise).            |
| Entité           | PRIMARY KEY | PRIMARY KEY (id_equipe)                 | Identifie de façon unique chaque enregistrement. |
| Domaine          | UNIQUE      | UNIQUE (nom_equipe)                     | Empêche deux équipes d'avoir le même nom.        |

## NOTIFICATION

| Type d'intégrité | Mécanisme | Contrainte                                                                   | Sens                                  |
|------------------|-----------|------------------------------------------------------------------------------|---------------------------------------|
| Domaine          | NOT NULL  | NOT NULL sur: date_heure, id_utilisateur, message, statut, type_notification | Champs obligatoires (valeur requise). |

| Type d'intégrité | Mécanisme   | Contrainte                                                                                   | Sens                                                                                                                      |
|------------------|-------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Entité           | PRIMARY KEY | PRIMARY KEY (id_notification)                                                                | Identifie de façon unique chaque enregistrement.                                                                          |
| Domaine / Métier | CHECK       | CONSTRAINT ck_notif_statut CHECK (statut IN ('LU', 'NON_LU'))                                | Limite le statut de lecture des notifications à une liste autorisée.                                                      |
| Référentielle    | FOREIGN KEY | CONSTRAINT fk_notif_user FOREIGN KEY (id_utilisateur) REFERENCES utilisateur(id_utilisateur) | Empêche notification.id_utilisateur de référencer une valeur inexistante (doit exister dans utilisateur(id_utilisateur)). |

## PROJET

| Type d'intégrité | Mécanisme   | Contrainte                                                                                                              |
|------------------|-------------|-------------------------------------------------------------------------------------------------------------------------|
| Domaine          | NOT NULL    | NOT NULL sur: budget_alloue, date_debut, date_fin_prevue, description_projet, id_equipe, priorite, statut, titre_projet |
| Entité           | PRIMARY KEY | PRIMARY KEY (id_projet)                                                                                                 |
| Domaine          | DEFAULT     | DEFAULT 0 sur budget_alloue                                                                                             |
| Domaine / Métier | CHECK       | CONSTRAINT ck_projet_budget CHECK (budget_alloue >= 0)                                                                  |
| Domaine / Métier | CHECK       | CONSTRAINT ck_projet_dates CHECK (date_fin_prevue >= date_debut)                                                        |
| Domaine / Métier | CHECK       | CONSTRAINT ck_projet_statut CHECK (statut IN ('EN_PLANIFICATION', 'EN_COURS', 'EN_PAUSE', 'TERMINE'))                   |
| Domaine / Métier | CHECK       | CONSTRAINT ck_projet_priorite CHECK (priorite IN ('HAUTE', 'MOYENNE', 'BASSE'))                                         |
| Référentielle    | FOREIGN KEY | CONSTRAINT fk_projet_equipe FOREIGN KEY (id_equipe) REFERENCES equipe(id_equipe)                                        |

## TACHE

| Type d'intégrité | Mécanisme   | Contrainte                                                                                                                     | Sens                                                                                               |
|------------------|-------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Domaine          | NOT NULL    | NOT NULL sur: id_projet, pourcentage_avancement, priorite, statut, titre_tache                                                 | Champs obligatoires (valeur requise)                                                               |
| Entité           | PRIMARY KEY | PRIMARY KEY (id_tache)                                                                                                         | Identifie de façon unique chaque enregistrement                                                    |
| Domaine          | DEFAULT     | DEFAULT 0 sur pourcentage_avancement                                                                                           | Valeur par défaut appliquée si n'est pas fournie.                                                  |
| Domaine / Métier | CHECK       | CONSTRAINT ck_tache_avancement CHECK (pourcentage_avancement BETWEEN 0 AND 100)                                                | Force l'avancement d'une tâche à rester entre 0 et 100.                                            |
| Domaine / Métier | CHECK       | CONSTRAINT ck_tache_dates CHECK (date_fin_prevue IS NULL OR date_debut_prevue IS NULL OR date_fin_prevue >= date_debut_prevue) | Empêche une date de fin prévue antérieure à la date de début si les deux dates sont renseignées).  |
| Domaine / Métier | CHECK       | CONSTRAINT ck_tache_statut CHECK (statut IN ('A_FAIRE', 'EN_COURS', 'TERMINE', 'BLOQUE'))                                      | Limite les statuts de tâche à une liste autorisée.                                                 |
| Domaine / Métier | CHECK       | CONSTRAINT ck_tache_priorite CHECK (priorite IN ('HAUTE', 'MOYENNE', 'BASSE'))                                                 | Limite les priorités de tâche à une liste autorisée.                                               |
| Référentielle    | FOREIGN KEY | CONSTRAINT fk_tache_projet FOREIGN KEY (id_projet) REFERENCES projet(id_projet)                                                | Empêche tache.id_projet de référencer une valeur inexistante (doit exister dans projet(id_projet)) |

## UTILISATEUR

| Type d'intégrité | Mécanisme   | Contrainte                                                                                               | Sens                                  |
|------------------|-------------|----------------------------------------------------------------------------------------------------------|---------------------------------------|
| Domaine          | NOT NULL    | NOT NULL sur: date_inscription, email_utilisateur, nom_utilisateur, prenom_utilisateur, role_utilisateur | Champs obligatoires (valeur requise). |
| Entité           | PRIMARY KEY | PRIMARY KEY (id_utilisateur)                                                                             | Identifie de façon unique             |

| Type d'intégrité | Mécanisme | Contrainte                                                                             | Sens                                                  |
|------------------|-----------|----------------------------------------------------------------------------------------|-------------------------------------------------------|
|                  |           |                                                                                        | chaque enregistrement.                                |
| Domaine          | UNIQUE    | UNIQUE (email_utilisateur)                                                             | Empêche deux utilisateurs d'avoir le même email.      |
| Domaine / Métier | CHECK     | CONSTRAINT ck_user_role CHECK (role_utilisateur IN ('ADMIN', 'CHEF_PROJET', 'MEMBRE')) | Limite les rôles d'utilisateur à une liste autorisée. |

## Requêtes SQL d'exploitation

### 10 requêtes simples

- 1. Lister tous les utilisateurs `SELECT * FROM UTILISATEUR;`
- 2. Afficher tous les projets en cours `SELECT * FROM PROJET WHERE statut = 'EN_COURS';`
- 3. Lister les tâches par ordre de priorité (Haute en premier) `SELECT * FROM TACHE ORDER BY CASE priorite WHEN 'HAUTE' THEN 1 WHEN 'MOYENNE' THEN 2 WHEN 'BASSE' THEN 3 END;`
- 4. Afficher les équipes créées récemment (ordre décroissant) `SELECT * FROM EQUIPE ORDER BY date_creation DESC;`
- 5. Lister les documents de type PDF `SELECT * FROM DOCUMENT WHERE type_fichier LIKE '%pdf%';`
- 6. Afficher les notifications non lues `SELECT * FROM NOTIFICATION WHERE statut = 'NON_LU';`
- 7. Lister les projets avec un budget supérieur à 1 000 000 FCFA `SELECT * FROM PROJET WHERE budget_alloue > 1000000;`
- 8. Afficher les tâches terminées `SELECT * FROM TACHE WHERE statut = 'TERMINE';`
- 9. Lister les utilisateurs avec le rôle 'CHEF\_PROJET' `SELECT * FROM UTILISATEUR WHERE role_utilisateur = 'CHEF_PROJET';`
- 10. Afficher les activités des 7 derniers jours `SELECT * FROM ACTIVITE WHERE date_heure >= DATE_SUB(NOW(), INTERVAL 7 DAY);`

### 10 requêtes intermédiaires

- 11. Compter le nombre de membres par équipe `SELECT e.nom_equipe, COUNT(a.id_utilisateur) as nombre_membres FROM EQUIPE e LEFT JOIN APPARTENIR a ON e.id_equipe = a.id_equipe GROUP BY e.id_equipe, e.nom_equipe;`

- 12. Afficher les projets avec le nom de leur équipe `SELECT p.titre_projet, p.budget_alloue, p.statut, e.nom_equipe FROM PROJET p LEFT JOIN EQUIPE e ON p.id_equipe = e.id_equipe;`
- 13. Lister les tâches avec le nom du projet associé `SELECT t.titre_tache, t.statut, t.priorite, p.titre_projet FROM TACHE t INNER JOIN PROJET p ON t.id_projet = p.id_projet;`
- 14. Compter le nombre de tâches par projet `SELECT p.titre_projet, COUNT(t.id_tache) as nombre_taches FROM PROJET p LEFT JOIN TACHE t ON p.id_projet = t.id_projet GROUP BY p.id_projet, p.titre_projet;`
- 15. Calculer le budget total alloué par équipe `SELECT e.nom_equipe, SUM(p.budget_alloue) as budget_total FROM EQUIPE e LEFT JOIN PROJET p ON e.id_equipe = p.id_equipe GROUP BY e.id_equipe, e.nom_equipe;`
- 16. Afficher le pourcentage d'avancement moyen par projet `SELECT p.titre_projet, AVG(t.pourcentage_avancement) as avancement_moyen FROM PROJET p LEFT JOIN TACHE t ON p.id_projet = t.id_projet GROUP BY p.id_projet, p.titre_projet;`
- 17. Lister les utilisateurs avec leurs équipes `SELECT u.nom_utilisateur, u.prenom_utilisateur, e.nom_equipe, a.date_affectation FROM UTILISATEUR u LEFT JOIN APPARTENIR a ON u.id_utilisateur = a.id_utilisateur LEFT JOIN EQUIPE e ON a.id_equipe = e.id_equipe;`
- 18. Compter le nombre de notifications par utilisateur `SELECT u.nom_utilisateur, u.prenom_utilisateur, COUNT(n.id_notification) as nb_notifications FROM UTILISATEUR u LEFT JOIN NOTIFICATION n ON u.id_utilisateur = n.id_utilisateur GROUP BY u.id_utilisateur, u.nom_utilisateur, u.prenom_utilisateur;`
- 19. Afficher les documents liés aux projets avec le nom du projet `SELECT d.nom_fichier, d.type_fichier, p.titre_projet, d.date_upload FROM DOCUMENT d INNER JOIN PROJET p ON d.id_projet = p.id_projet;`
- 20. Lister les tâches assignées à chaque utilisateur `SELECT u.nom_utilisateur, u.prenom_utilisateur, t.titre_tache, t.statut FROM UTILISATEUR u INNER JOIN ASSIGNER a ON u.id_utilisateur = a.id_utilisateur INNER JOIN TACHE t ON a.id_tache = t.id_tache;`

## 10 requêtes avancées

- 21. Trouver les projets avec plus de 5 tâches `SELECT p.titre_projet, COUNT(t.id_tache) as nombre_taches FROM PROJET p LEFT JOIN TACHE t ON p.id_projet = t.id_projet GROUP BY p.id_projet, p.titre_projet HAVING COUNT(t.id_tache) > 5;`
- 22. Afficher les équipes ayant un budget total supérieur à 5 000 000 FCFA `SELECT e.nom_equipe, SUM(p.budget_alloue) as budget_total FROM EQUIPE e INNER JOIN PROJET p ON e.id_equipe = p.id_equipe GROUP BY e.id_equipe, e.nom_equipe HAVING SUM(p.budget_alloue) > 5000000;`
- 23. Lister les utilisateurs qui n'ont aucune tâche assignée `SELECT u.id_utilisateur, u.nom_utilisateur, u.prenom_utilisateur FROM UTILISATEUR u WHERE u.id_utilisateur NOT IN ( SELECT DISTINCT id_utilisateur FROM ASSIGNER );`

- 24. Trouver les projets sans aucune tâche `SELECT p.id_projet, p.titre_projet, p.statut FROM PROJET p WHERE p.id_projet NOT IN ( SELECT DISTINCT id_projet FROM TACHE );`
- 25. Afficher les tâches avec leur projet, équipe et utilisateurs assignés `SELECT t.titre_tache, t.statut, p.titre_projet, e.nom_equipe, u.nom_utilisateur, u.prenom_utilisateur FROM TACHE t INNER JOIN PROJET p ON t.id_projet = p.id_projet LEFT JOIN EQUIPE e ON p.id_equipe = e.id_equipe LEFT JOIN ASSIGNER a ON t.id_tache = a.id_tache LEFT JOIN UTILISATEUR u ON a.id_utilisateur = u.id_utilisateur;`
- 26. Calculer le taux d'avancement global par équipe `SELECT e.nom_equipe, AVG(t.pourcentage_avancement) as taux_avancement_moyen, COUNT(t.id_tache) as total_taches FROM EQUIPE e INNER JOIN PROJET p ON e.id_equipe = p.id_equipe INNER JOIN TACHE t ON p.id_projet = t.id_projet GROUP BY e.id_equipe, e.nom_equipe;`
- 27. Trouver les utilisateurs les plus actifs (avec le plus de tâches) `SELECT u.nom_utilisateur, u.prenom_utilisateur, COUNT(a.id_tache) as nb_taches FROM UTILISATEUR u INNER JOIN ASSIGNER a ON u.id_utilisateur = a.id_utilisateur GROUP BY u.id_utilisateur, u.nom_utilisateur, u.prenom_utilisateur ORDER BY nb_taches DESC LIMIT 5;`
- 28. Lister les projets en retard (date fin prévue dépassée et statut non terminé) `SELECT p.titre_projet, p.date_fin_prevue, p.statut, e.nom_equipe FROM PROJET p LEFT JOIN EQUIPE e ON p.id_equipe = e.id_equipe WHERE p.date_fin_prevue < CURDATE() AND p.statut != 'Terminé';`
- 29. Afficher le nombre de documents par type et par projet `SELECT p.titre_projet, d.type_fichier, COUNT(d.id_document) as nombre_documents FROM PROJET p LEFT JOIN DOCUMENT d ON p.id_projet = d.id_projet GROUP BY p.id_projet, p.titre_projet, d.type_fichier ORDER BY p.titre_projet, nombre_documents DESC;`
- 30. Statistiques complètes par équipe (membres, projets, tâches, budget) `SELECT e.nom_equipe, COUNT(DISTINCT a.id_utilisateur) as nb_membres, COUNT(DISTINCT p.id_projet) as nb_projets, COUNT(DISTINCT t.id_tache) as nb_taches, COALESCE(SUM(p.budget_alloue), 0) as budget_total FROM EQUIPE e LEFT JOIN APPARTENIR a ON e.id_equipe = a.id_equipe LEFT JOIN PROJET p ON e.id_equipe = p.id_equipe LEFT JOIN TACHE t ON p.id_projet = t.id_projet GROUP BY e.id_equipe, e.nom_equipe;`

## Ecrans

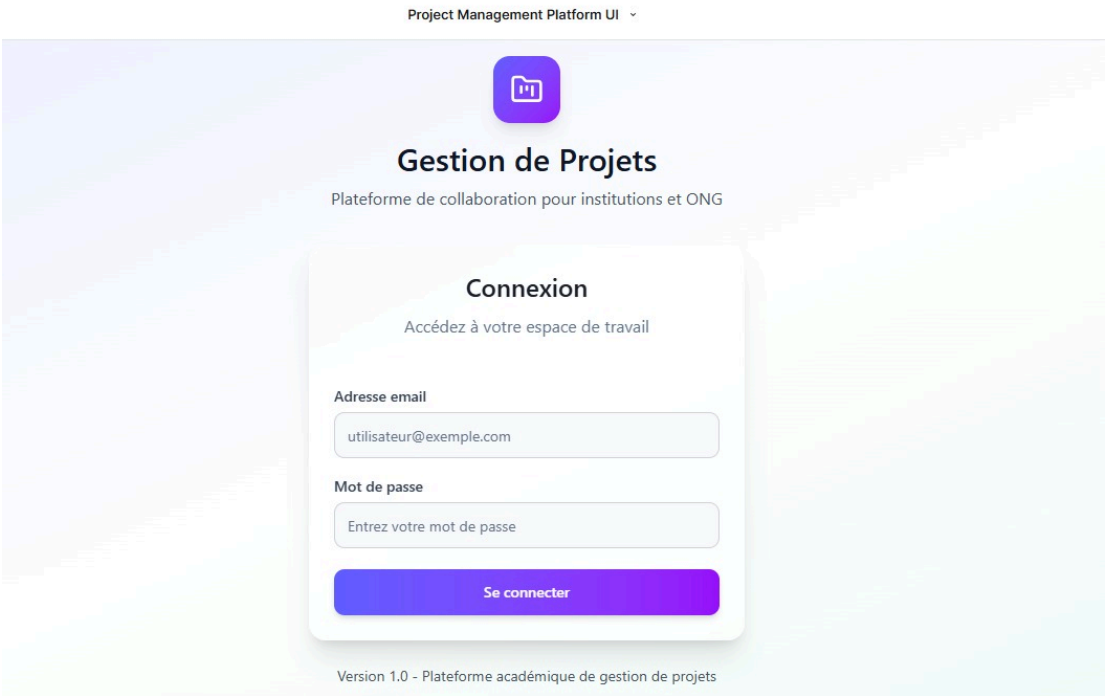
Ces écrans du logiciel ont été générés avec Figma.

## Présentation des maquettes de la plateforme

Cette section présente les interfaces utilisateurs conçues pour la plateforme ProjectFlow. Chaque écran a été pensé pour optimiser la gestion et le suivi des projets statistiques.

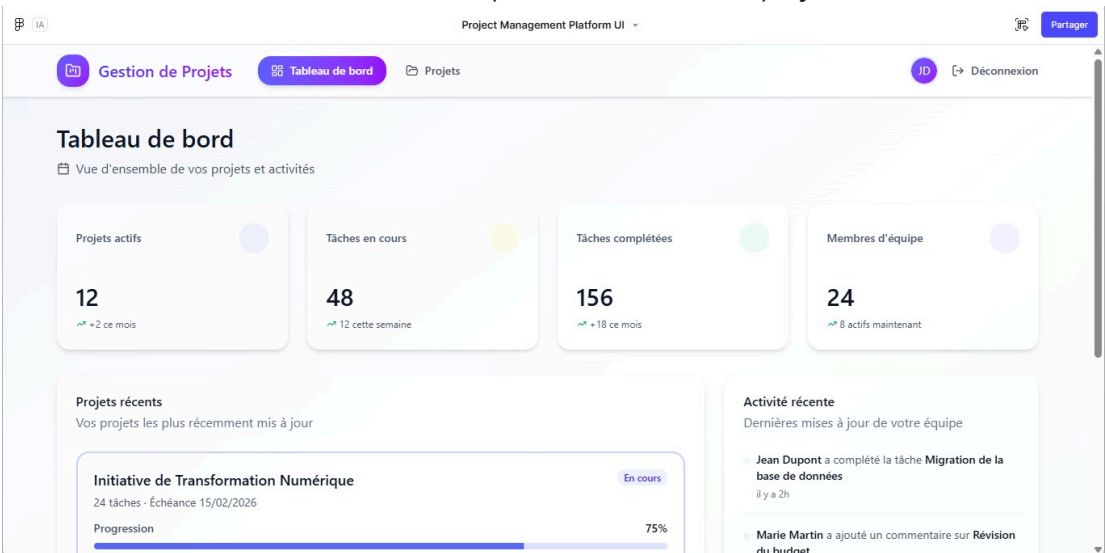
## Écran 1 : Connexion

L'interface de sécurisation permettant aux utilisateurs d'accéder à leur espace de travail.



## Écran 2 : Tableau de bord (Vue globale)

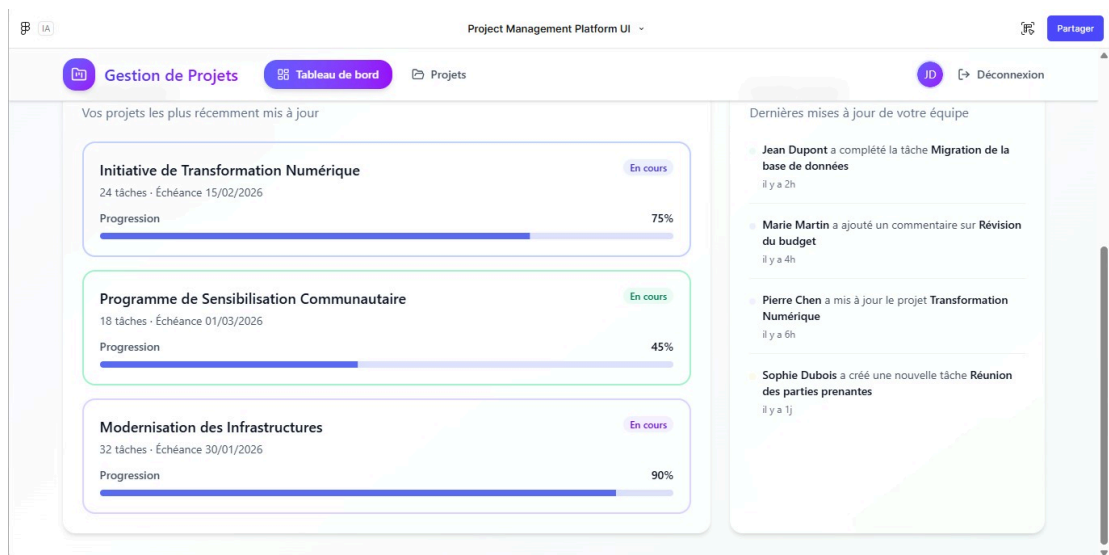
Vue d'ensemble des indicateurs clés de performance (KPI) du projet.



## Écran 3 : Activité de l'équipe

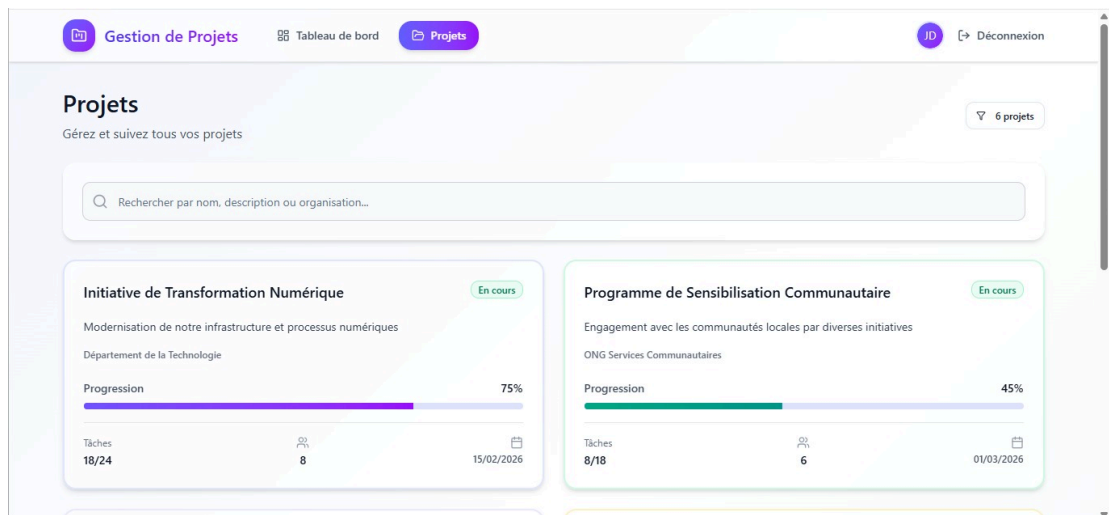
Suivi en temps réel des actions récentes effectuées par les membres de l'équipe.





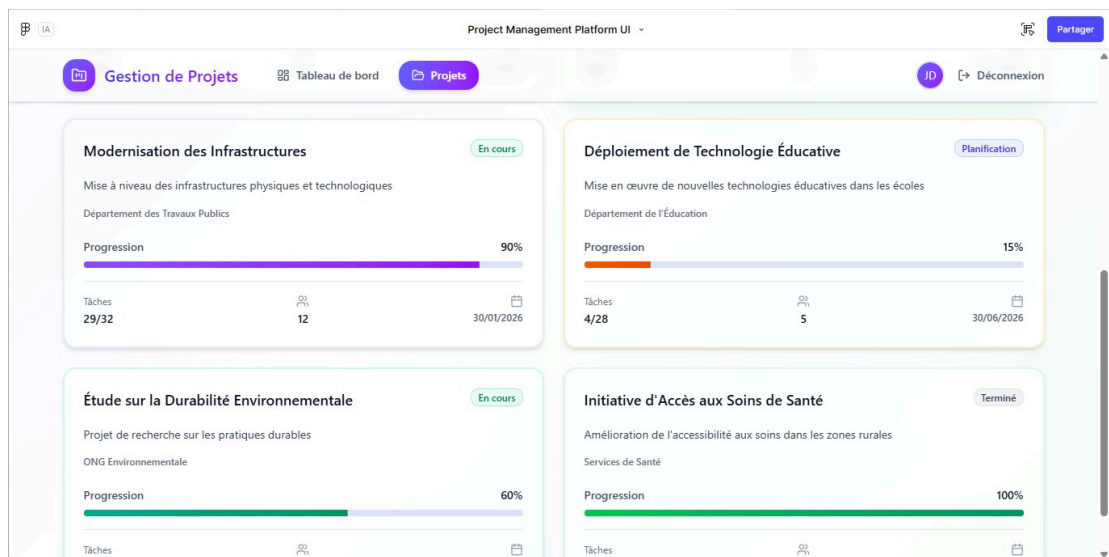
## Écran 4 : Liste détaillée des projets

Affichage exhaustif des projets avec leurs statuts respectifs.



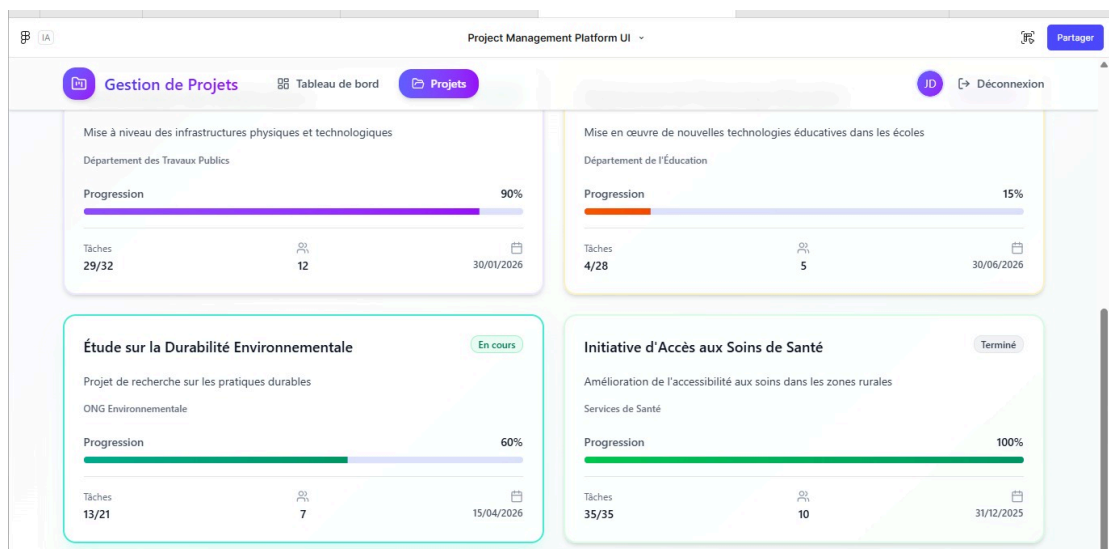
## Écran 5 : Vue synthétique

Une version condensée pour une lecture rapide des priorités.



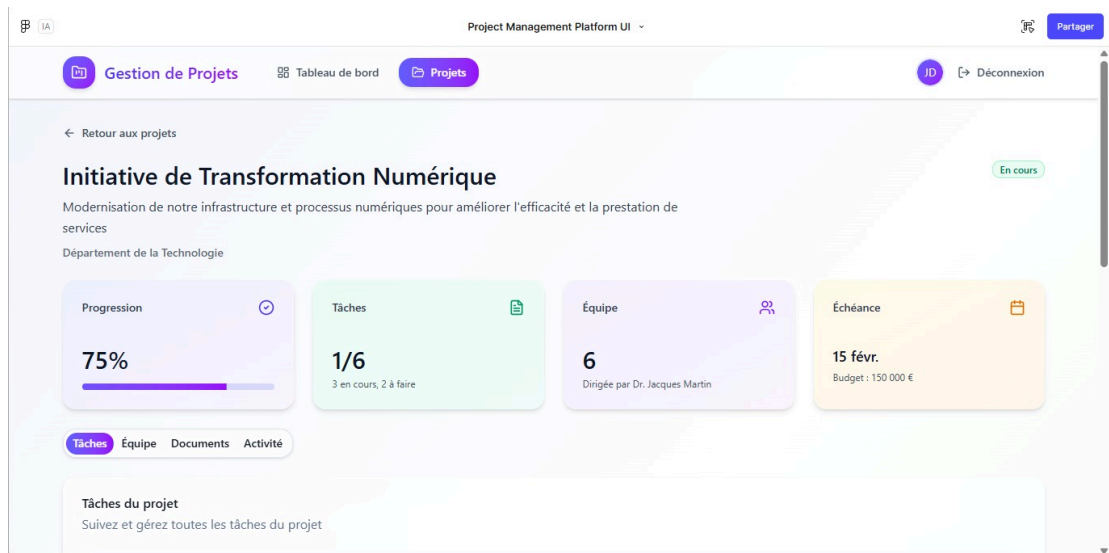
## Écran 6 : Navigation

Interface illustrant le système de pagination pour la gestion de gros volumes de données.



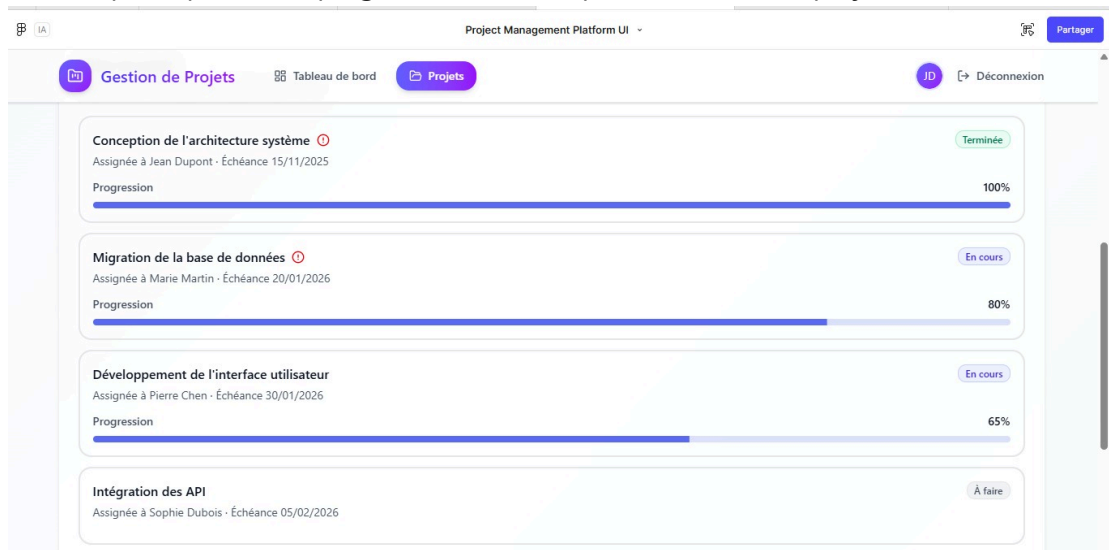
## Écran 7 : Statut des projets

Différenciation claire entre les projets en cours et ceux terminés.



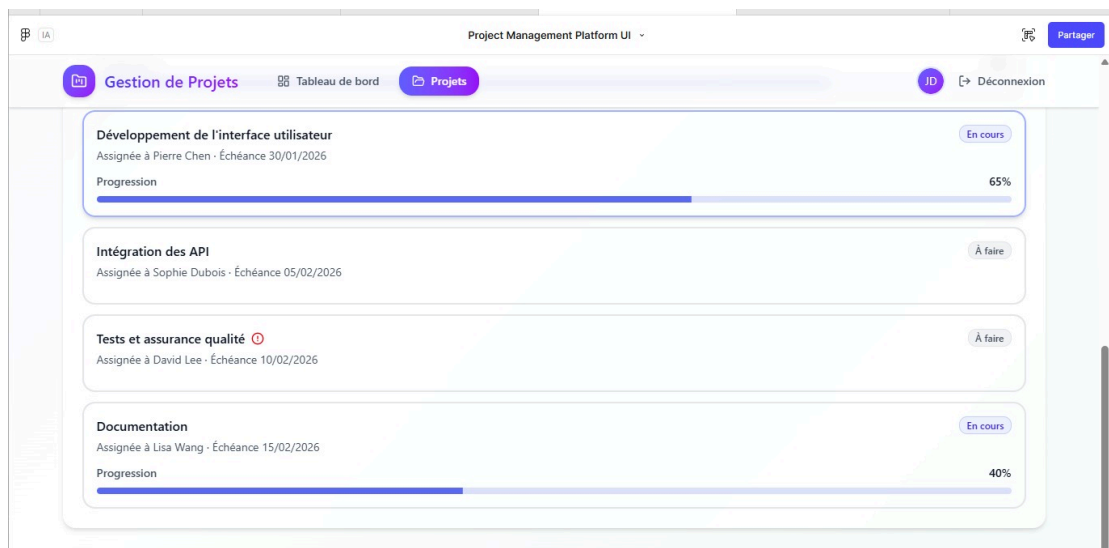
## Écran 8 : Vue d'ensemble du projet

Détails spécifiques sur la progression et les étapes franchies d'un projet sélectionné.



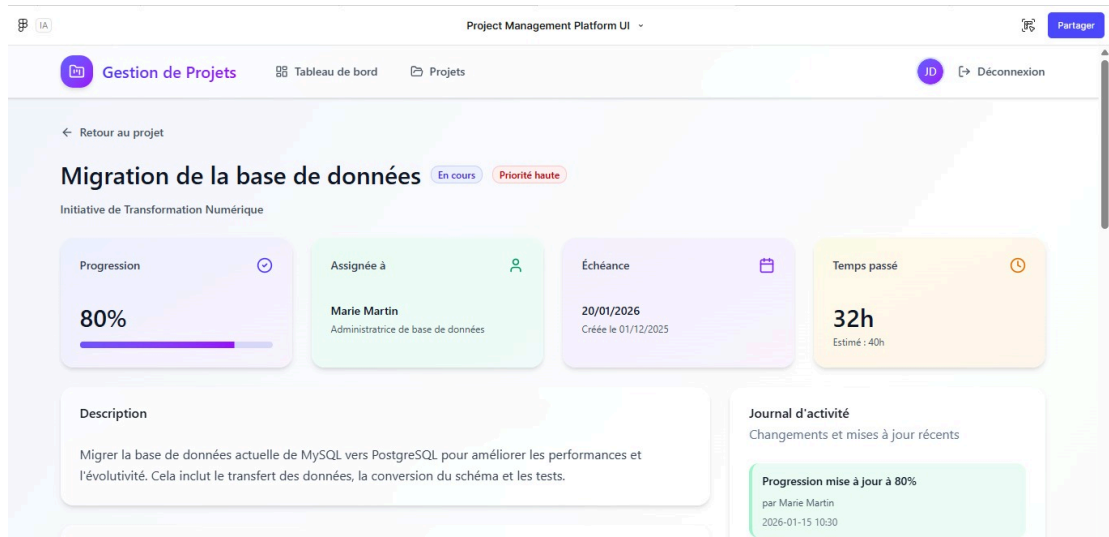
## Écran 9 : Liste des tâches

Décomposition d'un projet en unités de travail assignables.



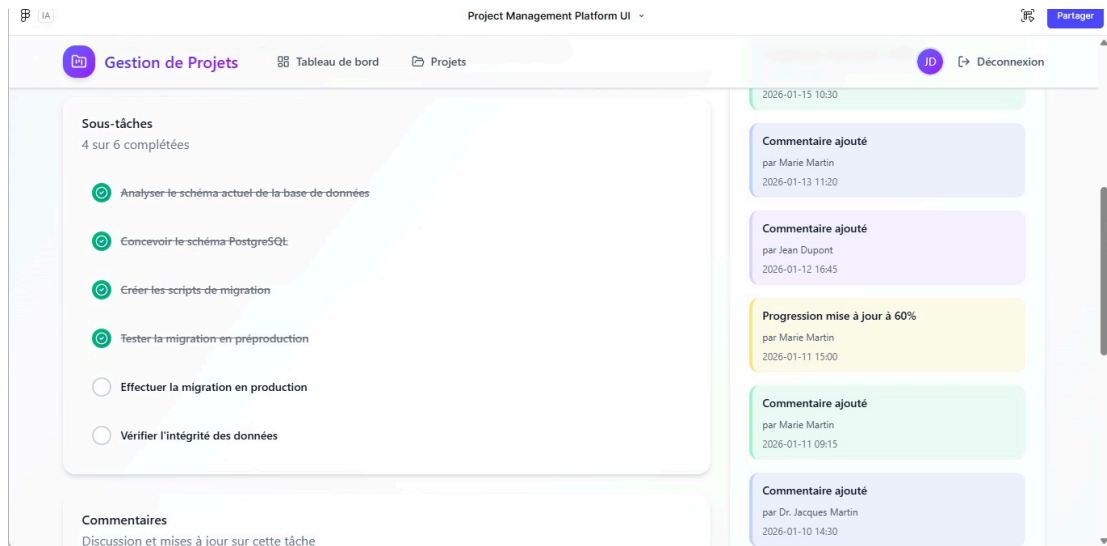
## Écran 10 : Gestion opérationnelle

Focus sur les tâches prioritaires à réaliser.



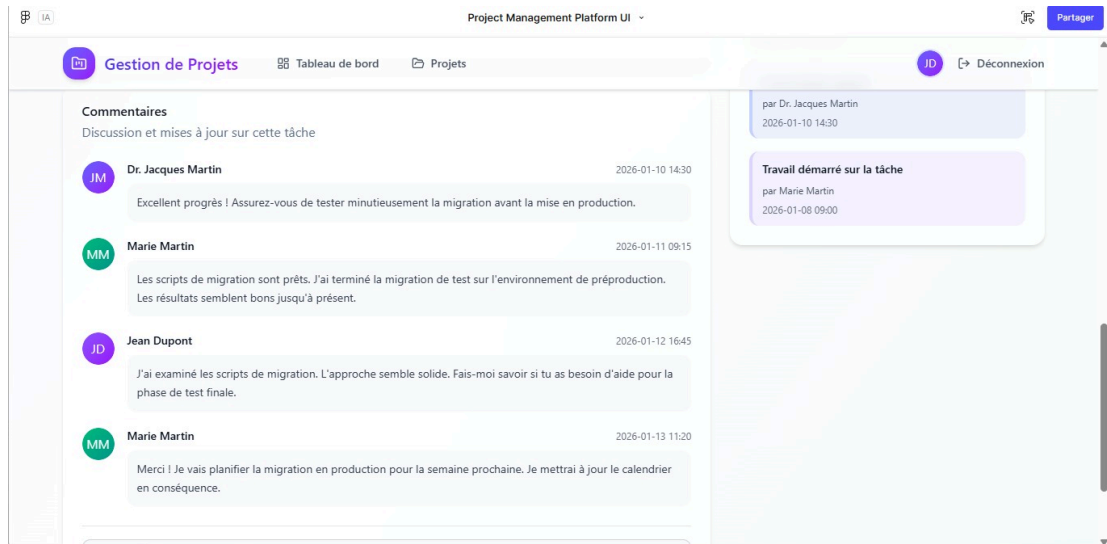
## Écran 11 : Sous-tâches et Journal

Historique détaillé des modifications et suivi des micro-tâches.



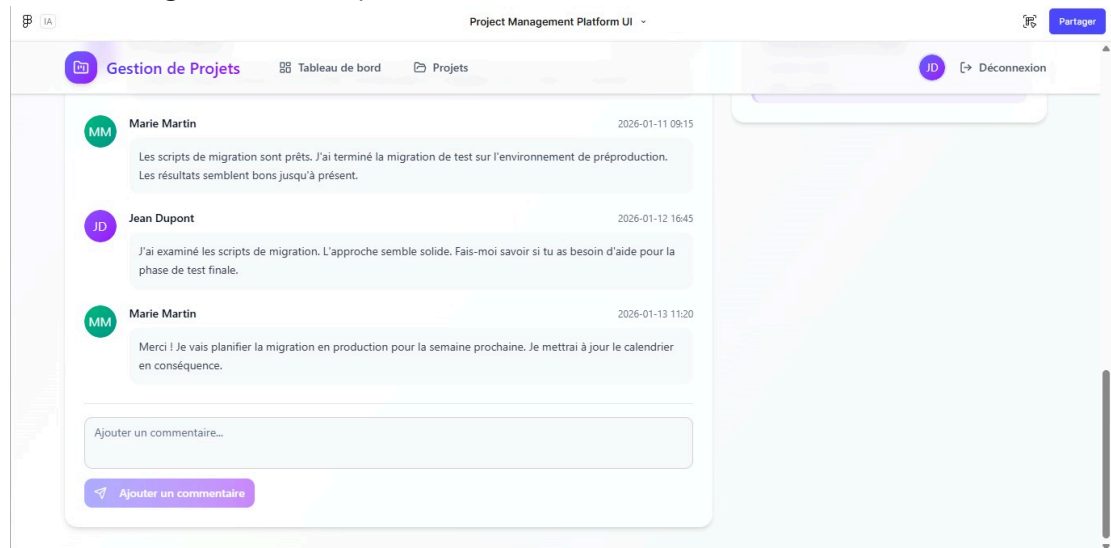
## Écran 12 : Collaboration

Espace dédié aux échanges entre les collaborateurs sur une tâche précise.



## Écran 13 : Suivi des commentaires

Interface de gestion et de réponse aux feedbacks des utilisateurs.



## Proposition d'amélioration de ProjectFlow

Bien que le modèle actuel couvre les fonctionnalités fondamentales de gestion de projet, plusieurs axes d'amélioration peuvent être envisagés pour accroître la performance et l'ergonomie de la plateforme :

- **Pilotage Visuel et Agilité** : L'intégration d'un système de **Kanban digital** permettrait une gestion dynamique des flux de travail, offrant aux utilisateurs une vue globale et intuitive de l'avancement des tâches en temps réel.
- **Analyse Prédictive et IA** : L'ajout d'une couche d'**intelligence artificielle** pourrait assister les chefs de projet dans la suggestion automatique de plannings et la prévision des retards potentiels en se basant sur l'historique des activités.
- **Gouvernance et Sécurité** : L'implémentation d'un module de **gestion des risques** permettrait d'identifier et de suivre les points critiques de chaque projet. De plus, un **système de validation** hiérarchique pour l'approbation des livrables renforcerait le contrôle qualité au sein des équipes.
- **Évaluation et Reporting** : Le modèle pourrait être étendu pour inclure l'**évaluation des membres** afin de mesurer la performance individuelle et collective, ainsi que la génération automatique de **rapports PDF** synthétisant l'état de santé financier et opérationnel des projets nationaux.

In [ ]: